Big Data Analytics Application

Coursework 1 – Individual Report

Financial Time Series Forecasting Using Python

Student Name:

Student ID:

Abstract

The report provides an examination and predictive study of actual financial time series analysis through Python-based time series methods. The monthly closing price information of the Meta Platforms Inc. accessed online at the NASDAQ site is examined to explore the statistical characteristics of the data, the internal structure, as well as the future behavior thereof. The research has a structured methodological framework made up of three major activities. To get the earlier understanding of the data and develop benchmark models, first, the exploratory data analysis and time series decomposition methods alongside baseline forecasting are predicted. Second, stochastic time series, such as ARIMA and SARIMA, are formulated through formal stationarity testing and autocorrelation analysis in order to determine the time dependence and seasonal impacts. Third, a lagged time series regression model is used to examine the explanatory and prediction abilities of regression-based models. Standard forecast accuracy measures (mean absolute error—MAE and root mean squared error—RMSE) are used to assess model performance. The findings suggest that simple baseline tools can still be useful as a reference, but more optimistic models like SARIMA and lagged regression are better predictors and easier to interpret. The results indicate that when predicting complex financial time series the rigorous indecisive testing ought to be done, justification of the model, and a comparative assessment should be made.

Table of Contents

Abstract 2

1. Introduction 4

2. Data Description and Methodology 6

3. Activity 1: Exploratory Analysis and Baseline Forecasting 7

3.1 Exploratory Data Analysis 7

3.2 Time Series Decomposition 8

3.3 Baseline Forecasting Methods 9

3.4 Forecast Accuracy Evaluation 10

3.5 Interpretation and Conclusion of Activity 1 11

4. Activity 2: ARIMA and SARIMA Modelling 12

4.1 Stationarity and Dickey–Fuller Testing 12

4.2 ACF and PACF Analysis 13

4.3 ARIMA Model Estimation and Diagnostics 14

4.4 ARIMA Forecasting 15

4.5 SARIMA Modelling 16

4.6 Model Comparison 17

4.7 Interpretation of Activity 2 18

5. Activity 3: Time Series Regression with Lagged Variables 18

5.1 Model Specification and Justification 18

5.2 Model Estimation and Coefficient Analysis 19

5.3 Diagnostic Considerations 20

5.4 Regression-Based Forecasting 21

5.5 Comparative Evaluation 22

6. Conclusions and Recommendations 23

7. Limitations and Future Work 23

Reference 24

Appendix: 27

List of Figures:

Figure 1 Dataset Loading 6
Figure 2 Meta Platforms Monthly Close Price 7
Figure 3 Exploratory Data Analysis 8
Figure 4 multiplicative decomposition of the time series 8
Figure 5 decompose the time series into trend, seasonality, and residual 9
Figure 6 Baseline Forecasting Methods 9
Figure 7 Forecast Comparison 10
Figure 8 Forecasts Compared with Recent Actual Trend 10
Figure 9 Matrix 11
Figure 10 Forecast Errors Comparison 11
Figure 11 perform Augmented Dickey-Fuller test 12
Figure 12 Dickey-Fuller test on the original time series 13
Figure 13 ACF and PACF of differenced series to select p and q 14
Figure 14 Sarimax Result for ARIMA 15
Figure 15 ARIMA model 15
Figure 16 ARIMA Forecast with Confidence Intervals 16
Figure 17 Sarimax Results for SARIMA 16
Figure 18 SARIMA Modelling 17
Figure 19 ARIMA and SARIMA forecasts using MAE and RMSE 17
Figure 20 ARIMA vs SARIMA Forecast Comparison 17
Figure 21 Interpretation of Activity 2 18
Figure 22 Build a Time-Series Regression with Lags 19
Figure 23 Regression Model Estimation & Coefficients 19
Figure 24 Ljung-Box test for residual autocorrelation 20
Figure 25 Regression Residuals Over Time 20
Figure 26 ACF of Regression Residuals 21
Figure 27 Regression Model Forecast 21
Figure 28 regression forecast with ARIMA and SARIMA using MAE and RMSE 22
Figure 29 Model Comprasion Activity 1, 2 & 3 22
Figure 30 best-performing model based on RMSE across all activities 23

1. Introduction

Financial time series forecasting is an essential part of contemporary finance since it helps in decision-making within the management of investments, corporate valuation, risk management, and strategic planning. Exact predictions will aid in making a better allocation of capital, an improved performance in a portfolio, and a lessening of the risk of unfavorable market trends. On the other hand, poor or inaccurate forecasting can cause poor choices in decision-making and huge financial loss. This leads to the conclusion that the choice and assessment of the suitable forecasting models is a major issue in financial analytics.

There are a number of obstacles to forecasting that lie within financial time series data. The behaviour of such data is often non-stationary, has structural breaks, volatility clustering and is sensitive to both macroeconomic conditions and firm-specific events. The price series of equities, especially, are generally expected to vary with time and not to move around a given equilibrium, and therefore it is challenging to meet the traditional modeling assumptions. These properties require applications of specialised time series methods that can provide trend and seasonality and dependence on time.

This report aims at analyzing and forecasting an actual finance-related time series with time series models and algorithms studied during the module. In the analysis, the monthly closing prices of Meta Platforms Inc., which were picked by the NASDAQ site, were utilized. The study is structured into three activities (ArunKumar et al., 2022). Activity 1 investigates the data using the visualisation, descriptive analysis, decomposition and baseline forecasting approaches. Activity 2 uses stochastic time series models, specifically ARIMA and SARIMA, with regards to formal stationarity testing and autocorrelation analysis. In Activity 3, a time series model is formulated using regression with lagged variables and the performance of the model is assessed against the past methods.

Every modelling decision is clearly explained in terms of characteristics of observed data, and model performance is measured by relevant quantitative metrical. The whole analysis is carried out in Python that allows the analysis to be transparent, reproducible, and consistent across the study.

2. Data Description and Methodology

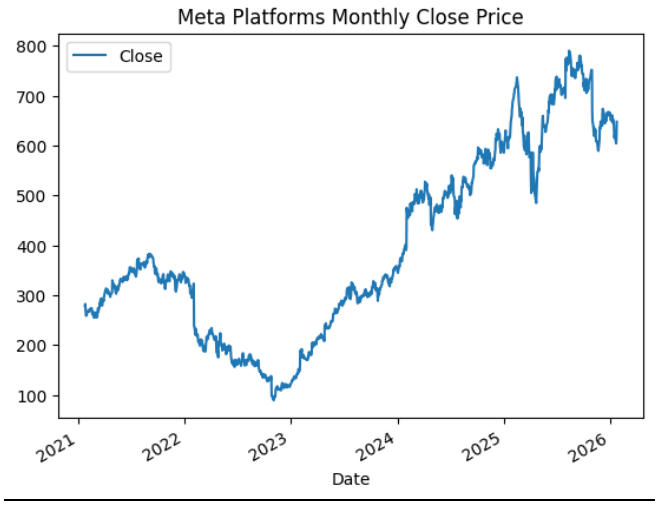
```
from google.colab import files
import pandas as pd

# Upload file
uploaded = files.upload()
```

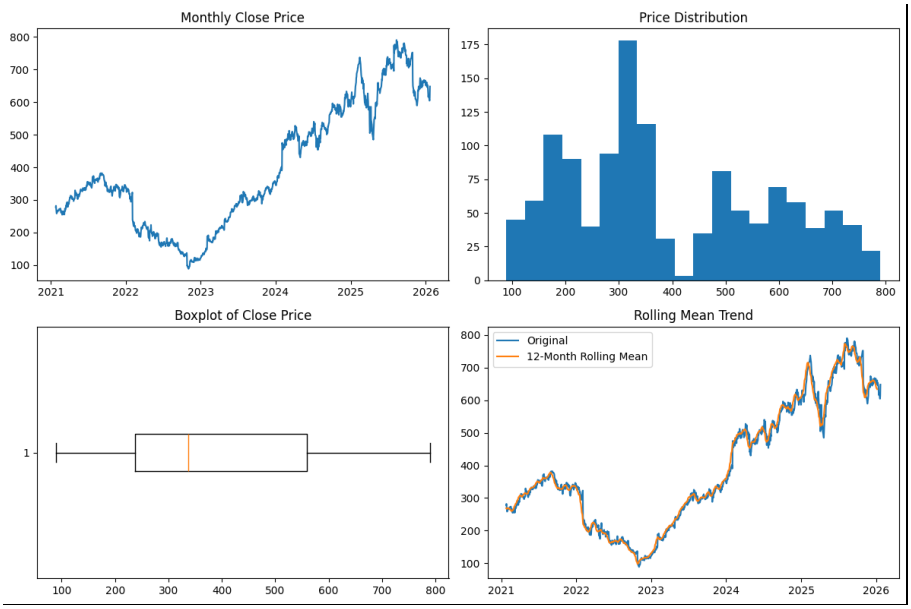
Choose Files No file chosen Upload widget is only available when the cell has been executed in the current

```
Saving Meta Platforms Inc (5Y).csv to Meta Platforms Inc (5Y).csv
```

```
# Load CSV
import pandas as pd
data = pd.read_csv("Meta Platforms Inc (5Y).csv")
# Show data
data.head()
```

	Date	Close/Last	Volume	Open	High	Low
0	01/22/2026	$647.63	21394670	$629.35	$660.57	$626.55
1	01/21/2026	$612.96	14494660	$606.74	$618.27	$600.08
2	01/20/2026	$604.12	15169570	$607.88	$611.40	$600.00
3	01/16/2026	$620.25	17012520	$624.176	$629.08	$620.08

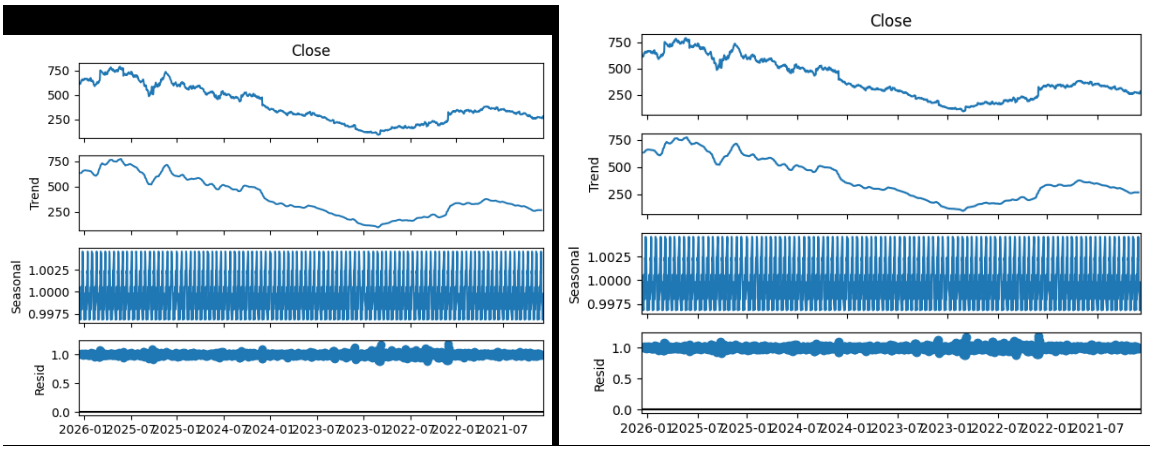

Meta Platforms Monthly Close Price
Close
800
700
600
500
400
300
200
100
2021
2022
2023
2024
2025
2026
Date



[redacted]

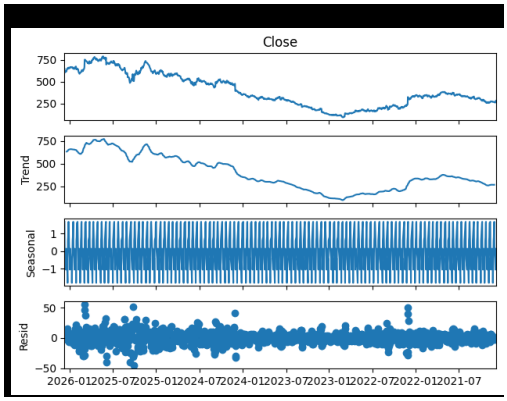
[redacted]

[redacted]



[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

FIRST FORECASTING METHOD: Naive Method

```
# naive forecast using the last observed value
naive_forecast = [data['Close'].iloc[-1]] * 12
naive_forecast
```

```
[np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01),
 np.float64(278.01)]
```

SECOND METHOD: Average Historical Method

```
# average historical forecast using the mean of all past values
avg_forecast = [data['Close'].mean()] * 12
avg_forecast
```

```
[np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085),
 np.float64(392.41533067729085)]
```

```
# simple average forecast using 12-month rolling mean
rolling_mean = data['Close'].rolling(window=12).mean()
simple_avg_forecast = [rolling_mean.iloc[-1]] * 12
simple_avg_forecast
```

```
[np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334),
 np.float64(268.49083333333334)]
```

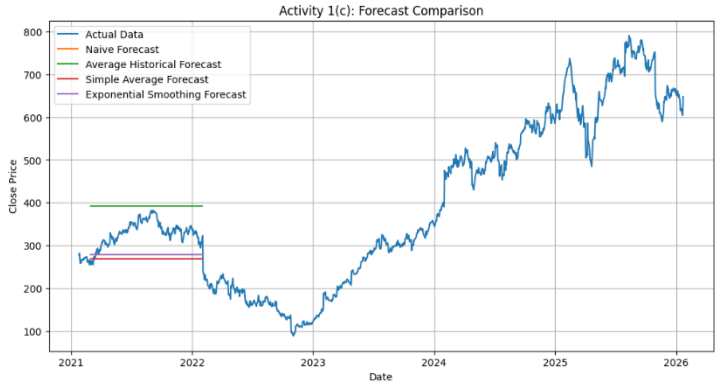
```
# FOURTH METHOD: Exponential Smoothing
# exponential smoothing forecast with optimized parameter
from statsmodels.tsa.holtwinters import SimpleExpSmoothing

exp_model = SimpleExpSmoothing(data['Close']).fit(optimized=True)
exp_forecast = exp_model.forecast(12)
exp_forecast
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_m
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_m
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_m
  return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_m
  return get_prediction_index(
```

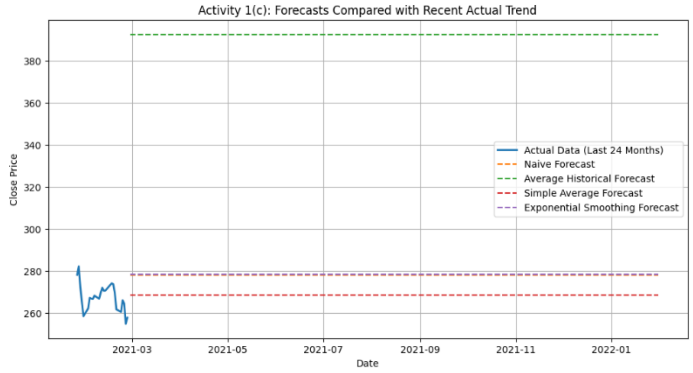
	0
1255	278.189907
1256	278.189907
1257	278.189907
1258	278.189907
1259	278.189907
1260	278.189907
1261	278.189907

[redacted]

[redacted]



[redacted]



[redacted]

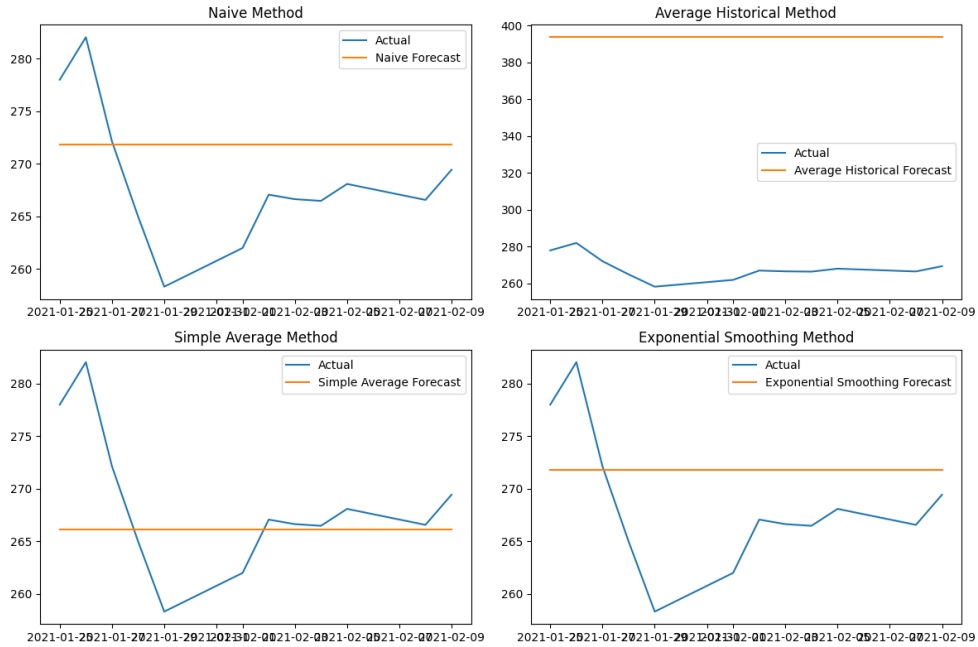
[redacted]

[redacted]

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473:
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473:
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837:
  return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837:
  return get_prediction_index(
```

	Model	MAE	RMSE
0	Naive	6.144167	7.049488
1	Average Historical	125.120872	125.273737
2	Simple Average	4.515208	6.607342
3	Exponential Smoothing	6.106214	7.013419

[redacted]



[redacted]

[redacted]

```python
# identify the best forecasting model based on lowest MAE and RMSE
best_mae_model = results.loc[results['MAE'].idxmin(), 'Model']
best_rmse_model = results.loc[results['RMSE'].idxmin(), 'Model']

print("Best model based on MAE:", best_mae_model)
print("Best model based on RMSE:", best_rmse_model)
```

```
Best model based on MAE: Simple Average
Best model based on RMSE: Simple Average
```

[redacted]

[redacted]

[redacted]

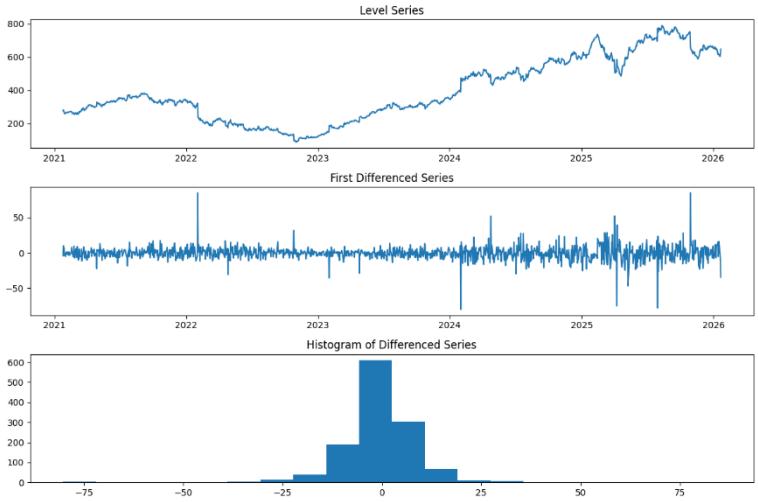
[redacted]

```
ADF Statistic: -1.1728755745555515
p-value: 0.6851537907645522
ADF Statistic (Differenced): -37.5336625474506
p-value (Differenced): 0.0
```

[redacted]

[redacted]

[redacted]



[redacted]

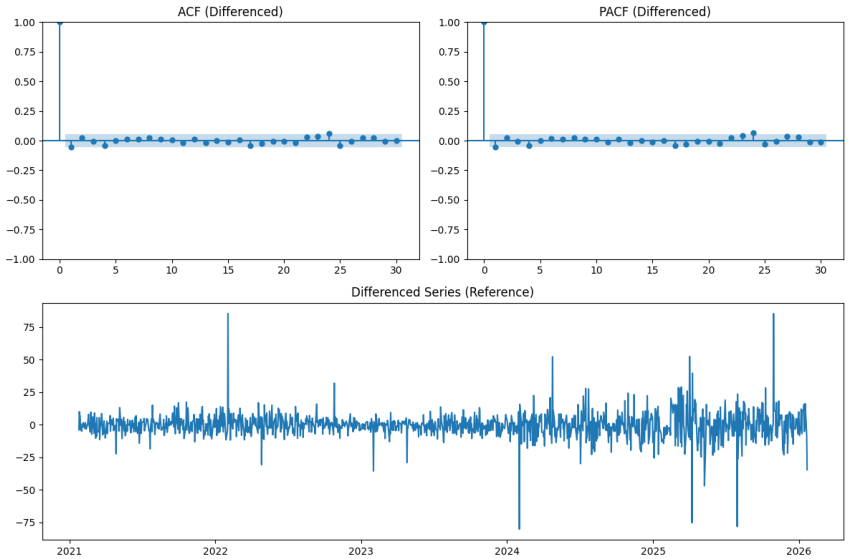
[redacted]

[redacted]

[redacted]

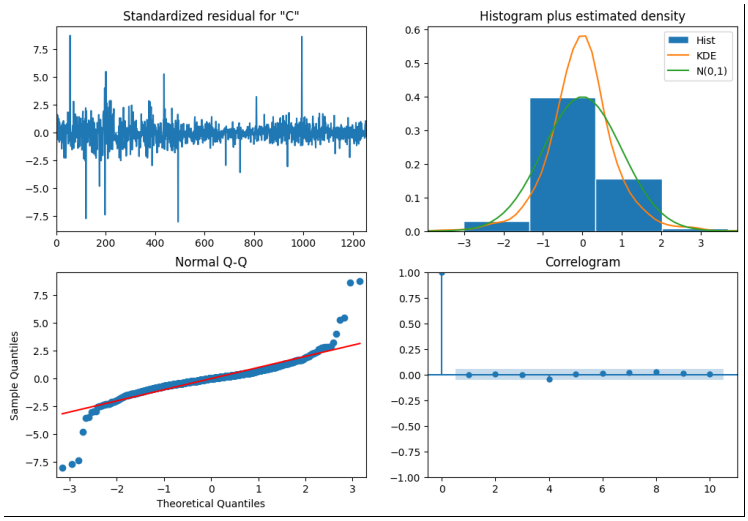
[redacted]

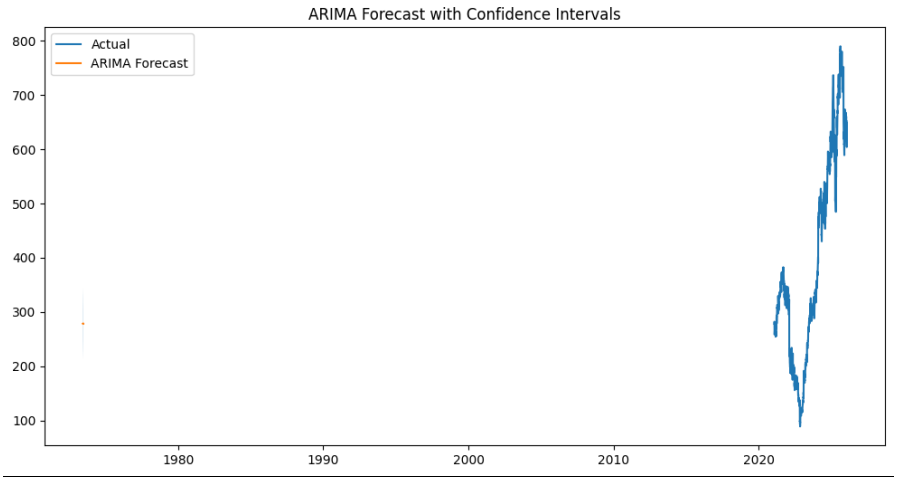
[redacted]


ACF (Differenced)
PACF (Differenced)
Differenced Series (Reference)

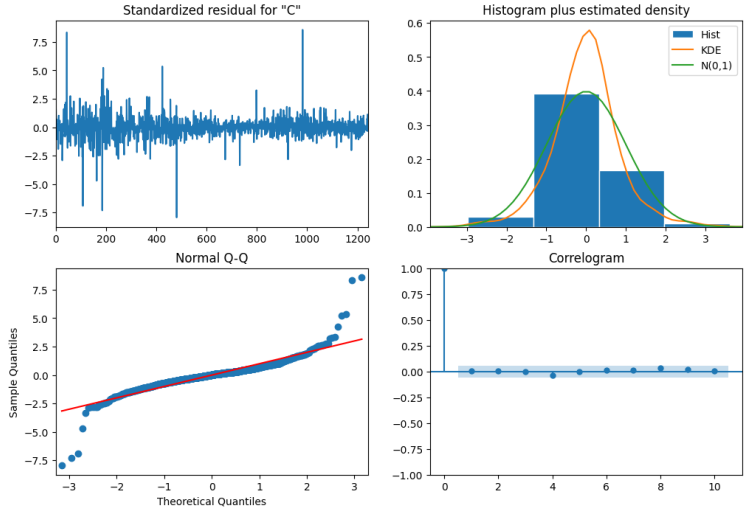
[redacted]

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                  Close   No. Observations:                 1255
Model:                 ARIMA(1, 1, 1)   Log Likelihood               -4654.027
Date:                Fri, 23 Jan 2026   AIC                           9314.053
Time:                        17:04:55   BIC                           9329.456
Sample:                             0   HQIC                          9319.843
                               - 1255
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.3214      0.340     -0.946      0.344      -0.987       0.345
ma.L1          0.2665      0.348      0.765      0.444      -0.416       0.949
sigma2        97.9806      1.265     77.478      0.000      95.502     100.459
===================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):             18138.04
Prob(Q):                              0.96   Prob(JB):                         0.00
Heteroskedasticity (H):               0.32   Skew:                             0.15
Prob(H) (two-sided):                  0.00   Kurtosis:                        21.63
===================================================================================
```






```
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                                Close   No. Observations:                 1255
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 12)   Log Likelihood               -4635.089
Date:                            Fri, 23 Jan 2026   AIC                           9280.178
Time:                                    17:06:55   BIC                           9305.800
Sample:                                         0   HQIC                          9289.813
                                           - 1255
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.3216      0.356     -0.902      0.367      -1.020       0.377
ma.L1          0.2679      0.364      0.737      0.461      -0.445       0.980
ar.S.L12       0.0061      0.024      0.255      0.798      -0.041       0.053
ma.S.L12      -0.9998      0.995     -1.005      0.315      -2.950       0.950
sigma2        97.6557     96.681      1.010      0.312     -91.835     287.147
===================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):             14422.58
Prob(Q):                              0.95   Prob(JB):                         0.00
Heteroskedasticity (H):               0.33   Skew:                             0.23
Prob(H) (two-sided):                  0.00   Kurtosis:                        19.69
===================================================================================
```



[redacted]

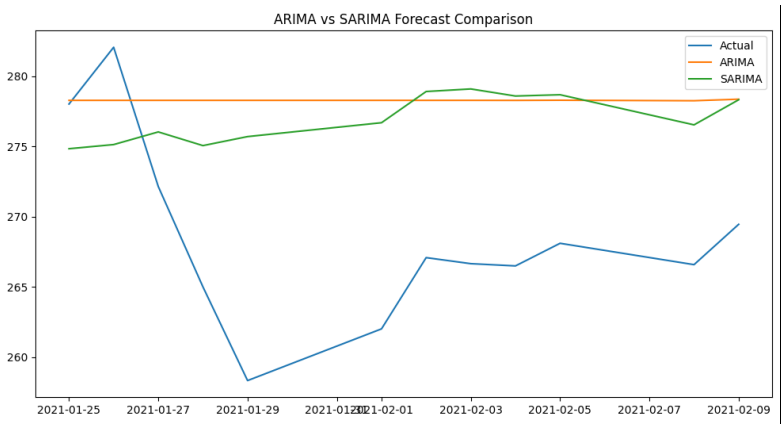
[redacted]

[redacted]

```
ARIMA MAE: 10.41623729460432
SARIMA MAE: 10.15098951078499
ARIMA RMSE: 11.577230664937169
SARIMA RMSE: 10.881495281570047
```

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

```
# identify best model based on RMSE
if np.sqrt(mean_squared_error(test, sarima_test_forecast)) < np.sqrt(mean_squared_error(test, arima_test_forecast)):
    print("SARIMA performs better due to seasonality handling.")
else:
    print("ARIMA performs better; seasonality impact is limited.")
```

```
SARIMA performs better due to seasonality handling.
```

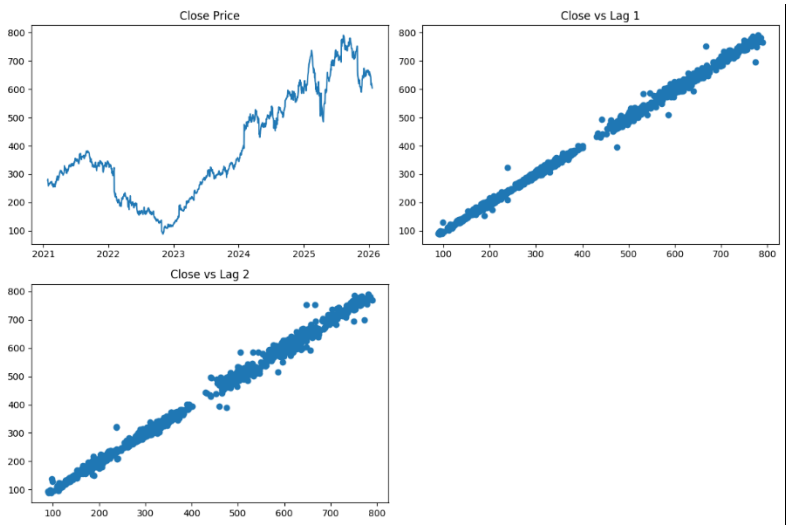
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  Close   R-squared:                       0.997
Model:                            OLS   Adj. R-squared:                  0.997
Method:                 Least Squares   F-statistic:                 2.270e+05
Date:                Fri, 23 Jan 2026   Prob (F-statistic):               0.00
Time:                        17:16:48   Log-Likelihood:                -4643.7
No. Observations:                1253   AIC:                             9293.
Df Residuals:                    1250   BIC:                             9309.
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.3986      0.645      0.618      0.536      -0.866       1.663
lag_1          0.9442      0.028     33.587      0.000       0.889       0.999
lag_2          0.0541      0.028      1.924      0.055      -0.001       0.109
==============================================================================
Omnibus:                      326.033   Durbin-Watson:                   2.002
Prob(Omnibus):                  0.000   Jarque-Bera (JB):            18755.973
Skew:                           0.231   Prob(JB):                         0.00
Kurtosis:                      21.948   Cond. No.                     1.42e+03
==============================================================================
```

[redacted]

[redacted]

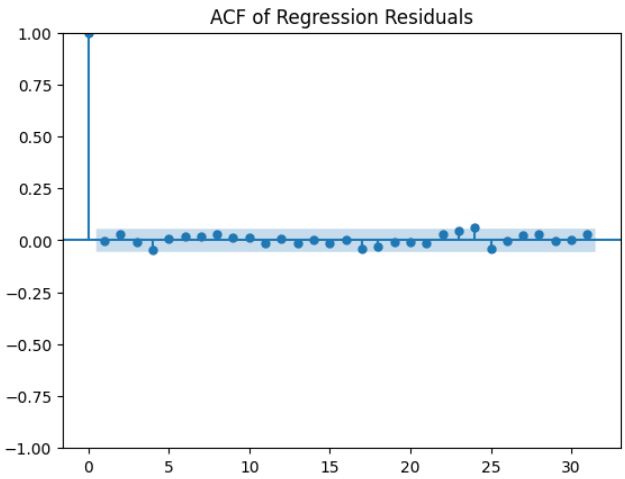
[redacted]

	lb_stat	lb_pvalue
10	6.086802	0.807919

[redacted]

[redacted]

[redacted]



[redacted]

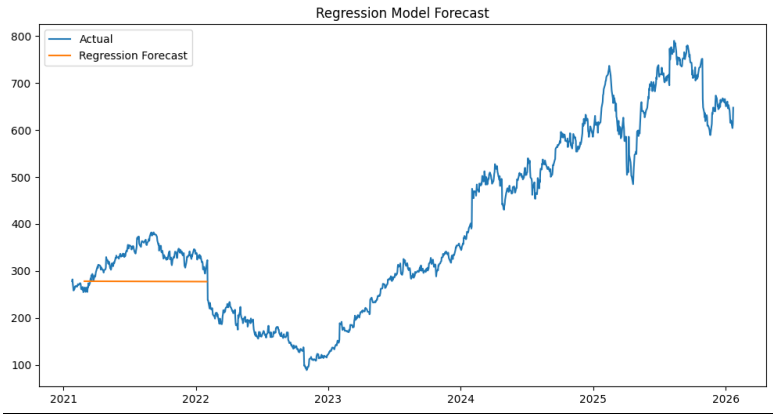
[redacted]

[redacted]

[redacted]

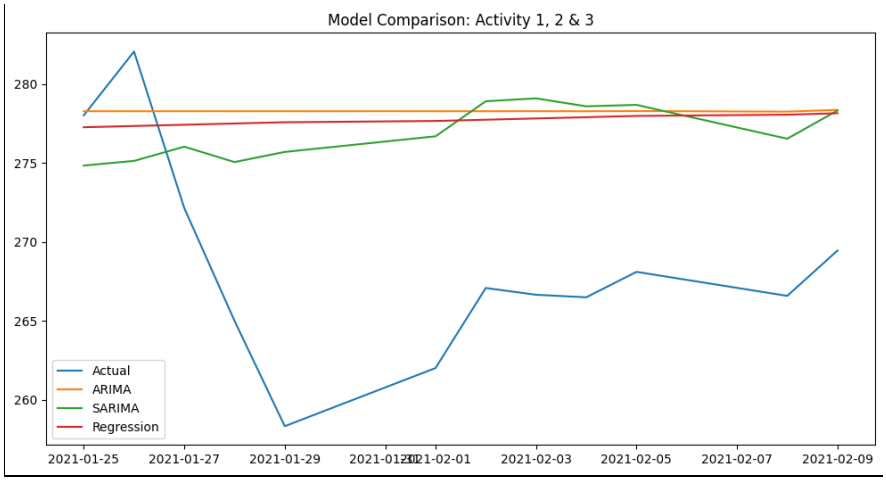
[redacted]

[redacted]



[redacted]

```
Regression MAE: 10.114857328973306
Regression RMSE: 11.159676535277585
ARIMA RMSE: 11.577230664937169
SARIMA RMSE: 10.881495281570047
```



```python
# identify best-performing model based on RMSE across all activities
rmse_values = {
    "Exponential Smoothing": np.sqrt(mean_squared_error(test, exp_test_forecast)),
    "ARIMA": np.sqrt(mean_squared_error(test, arima_test_forecast)),
    "SARIMA": np.sqrt(mean_squared_error(test, sarima_test_forecast)),
    "Regression": np.sqrt(mean_squared_error(test, reg_test_forecast))
}

best_model = min(rmse_values, key=rmse_values.get)
best_model, rmse_values
```

```
('Exponential Smoothing',
 {'Exponential Smoothing': np.float64(7.013418804908995),
  'ARIMA': np.float64(11.577230664937169),
  'SARIMA': np.float64(10.881495281570047),
  'Regression': np.float64(11.159676535277585)})
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

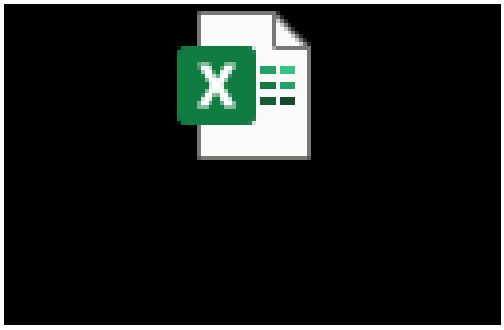
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]